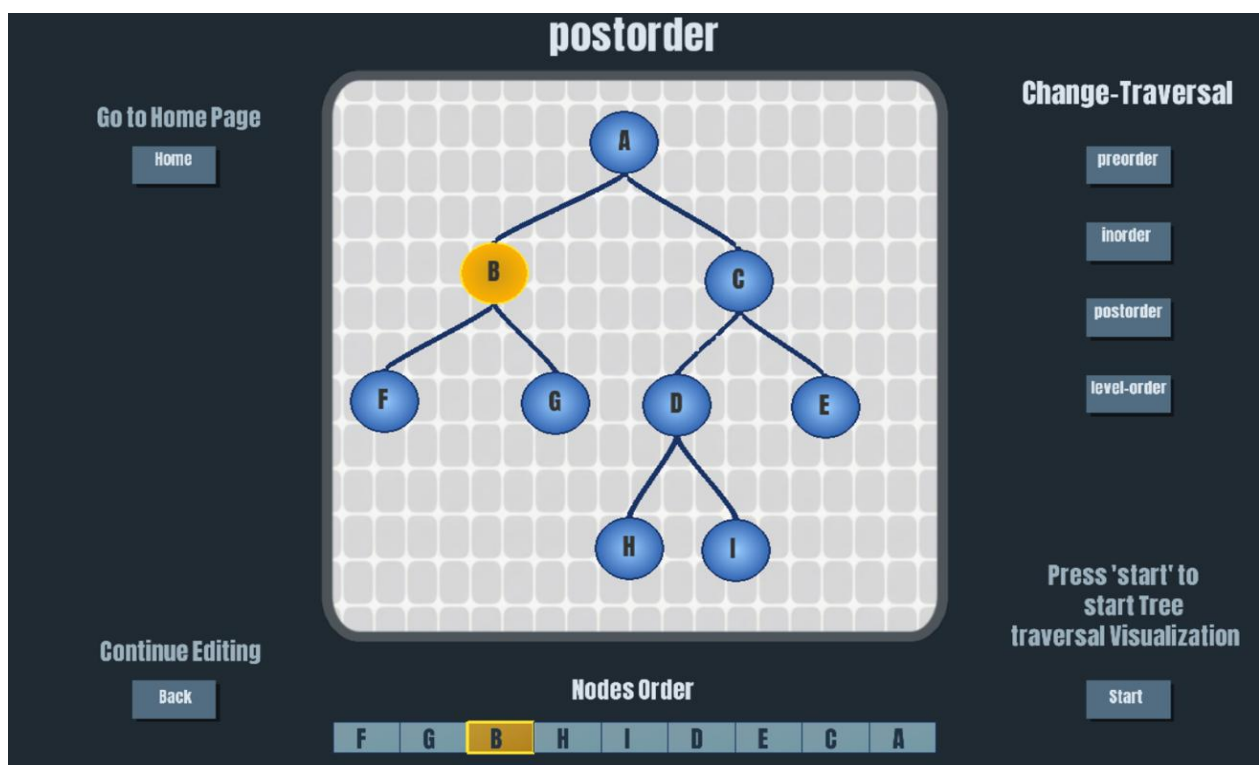


ATHANASIOS GOURDOMICHALIS
ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS
Assignment in Computer Programming with C++

General project presentation and documentation report

INTERACTIVE APPLICATION USING A NODE NETWORK:
REPRESENTATION OF TREE TRAVERSALS IN A USER-
CREATED BINARY TREE



1. Introduction – Brief Description

- The interactive application we created is based on the functionality of a network (graph) of nodes.
 - We implemented a simulation (educational application)
 - It is related to the field of **DATA STRUCTURES** (a course taught concurrently in the same semester).
 - **NOTE:** The utilized library (SGG) aims to provide a simple interface for students to include basic graphics, audio and interaction functionality in their applications written in C++.
- SGG offers an abstraction and simplification layer over SDL, OpenGL

and other core windowing, graphics and audio libraries, providing a clear, simple and hassle-free programming interface to build from simple interactive applications to 2D computer games and data visualization tools.

It was provided to us via GitHub: [cgaueb/sgg: Simple Game Graphics Library](https://github.com/cgaueb/sgg) and its installation and use were mandatory for our application's implementation. Nevertheless, **this documentation report does not provide a detailed description of the library's programmatic functions**, as we were only required to familiarize ourselves with the library and utilize it where necessary. **References to the specific library functions used are made solely within the technical documentation PDF (technical-report-code-analysis.pdf)** where the project's codebase is thoroughly explained. A detailed description and documentation of the library is available at the following link: [Simple Game Graphics Library: Main Page](#).

- The binary tree created by the user, strictly utilizes the SGG library ([Simple Game Graphics Library](#)) for window management, keyboard and mouse events, graphics rendering, and audio playback.

 There are 4 Traversals in total:

- **3 DFS (Depth-First Search) Traversals:**
 - Preorder: (Root - Left - Right)
 - Inorder: (Left - Root - Right)
 - Postorder: (Left - Right - Root)
- **1 BFS (Breadth-First Search) Traversal:**
 - Level-order: Level-by-level traversal (from top to bottom and from left to right)

2. Regarding the Potential Ideas (assignment.pdf)

- Just as the A* algorithm for a graph traversal / Dijkstra's algorithm is mentioned among the suggested ideas and being motivated by the fact that creativity is rewarded, we designed our own application:
- **A simulator for Binary Tree Traversals. A binary tree is technically a graph according to the fundamental principles of Data Structures.**

3. Regarding the implementation: Speed

- **PNG images** (visual assets of the application) for loading and drawing images on top of the basic primitives.
- Using an image editing and conversion program, we adjusted them to an appropriate size to sufficiently cover the expected window resolution (while preserving the aspect ratio of the original image).
- We converted them to **dimensions that are powers of 2** → for faster loading by the library, thereby avoiding unnecessary waste of: a) memory, b) opening time (especially in debug mode), and c) disk space and space in the zip archive.

✚ *The reason of having many code files (including both **.h** header files and **.cpp** source files) → 27 files in total:*

4. Regarding the assignment's optional features:

In the grading of the assignment, the following were positively evaluated:

a) The proper structuring of the codebase into corresponding header (.h) and implementation (.cpp) files.

b) METICULOUS METHOD DECLARATION: Proper use of references, const arguments, and const member functions.

c) The use of a template class (Node<T>):

- Initially, we intended to provide extended functionality by allowing the user to select the data type stored in a node (int, float, double, etc.) → due to time constraints, this was deferred in favor of other core features.

5. Use of dynamic memory:

- In our interactive application, entities (**GameObject**, **Level**, etc.) are instantiated and "live" for a limited duration.
- We instantiate them dynamically using **new** (heap allocation).
- We deallocate them using **delete**.

6. Utilization of standard STL containers and structures:

STL CONTAINERS: `vector`, `queue`, `stack`

- We use them for core program logic and operations (**`stack`**, **`queue`**).
- We use them for rendering graphics on the screen (**`vector`**).
- Thus, we select the most suitable data structures for optimal storage and data access.

7. According to section 8. "Assignment Submission" of assignment.pdf:

FOLDER HIERARCHY AND PROJECT STRUCTURE:

- **Folder TreeProject / start / TreeProject:** Assignment source code (along with .sln, .vcxproj, etc.)
- **Folder TreeProject / start / TreeProject / bin:** Executable
- **Folder bin / assets:** Audiovisual assets → structured so the built executable can locate them.
- **Folder 3rd party / SGG used components:**
 - / lib / .lib files
 - / include / 2 header files (**`graphics.h`**, **`scancodes.h`**)

TreeProject/

```
|__sgg-main/ } The SGG library installed upon downloading the whole ZIP from:
|              cgaueb/sgg: Simple Game Graphics Library
|
|__start/
|
|__TreeProject/
|
|__3rdParty/
|
|__include/
|
|__sgg/ - graphics.h } SGG header files
|      - scancodes.h
|
|__lib/ - sgg.lib } SGG Static Libraries files
|      - sggd.lib
|
|__bin/
|
|__assets/ - Audiovisual Material (.wav, .ttf, .png)
|          - .dll Dynamic Libraries, .pdb files from SGG
|          - .exe executable file
|
|__TreeProject/ } Visual Studio solution file
|              - .h header, .cpp source code files
|              - .vcxproj, .vcxproj.user, .vcxproj.user etc.
|
|__TreeProject.sln} Visual Studio solution
```

8. CLASS INHERITANCE (SYSTEM DESIGN – CLASS HIERARCHIES)

INTERNAL HIERARCHICAL STRUCTURE UML DIAGRAM

